

Lab 9

ELECENG 3EY4

Alaa Abdelsalam – abdelal74 – 400396373

Tina Ismail – ismait1 – 400299939

Marisa Patel – patem156 – 400393635

April 4, 2024

Table of Contents

Contribution	2
Activity 1	2
Activity 2	3
Activity 3	4
Activity 4	5
Activity 5	6
Activity 6	7
Activity 7	8
Activity 8	9
Activity 9	9
Activity 10	10
Activity 11	10
Activity 12	12
Activity 13	13
Activity 14	14
Activity 15	15
Activity 16	16
Activity 17	17
Activity 18	17
Activity 19	17
Activity 20	18

Contribution

Activities 1 to 20: Alaa, Tina, Marisa

Activity 1

In activity 1, we copied the contents of the incomplete “navigation_lab9_inc.py” file into our “navigation.py” file. Then, we updated our “params.yaml” file with the one posted on Avenue. In this file, we updated the calibrated VESC parameters that we obtained in Lab 6 as well as the “scan_beams” value. This can be seen in Figures 1 and 2 below. From Avenue, we also copied the “hallway_w_blocks.yaml” and “hallway_w_blocks.pgm” files to the “maps” subdirectory of the “fltenth_simulator” package. Then, we modified the “simulator.launch” file accordingly so we could load the map for the simulations.

```
# Ackermann to VESC parameters
speed_to_erpm_gain: 3182.18
speed_to_erpm_offset: 0.0
steering_angle_to_servo_gain: -0.823
steering_angle_to_servo_offset: 0.5000
driver_smoother_rate: 75.0 # messages/sec
```

Figure 1. Updated Calibrated VESC Parameters in the “params.yaml” File

```
# Lidar simulation parameters
scan_beams: 720
scan_field_of_view: 6.2831853 #4.71 # radians
scan_range: 12.0
```

Figure 2. Updated “scan_beams” Parameters in the “params.yaml” File

Activity 2

Parts of the code in the “navigation.py” and “params.yaml” files that accomplish step 1, which is defining a Field of View (FOV) in front of the vehicle where we look for potential obstacles. The parts of the code in the “navigation.py” file that accomplish step 1 include the “__init__,” “preprocess_lidar,” “find_max_gap,” and “lidar_callback” functions.

The “__init__” function initializes the parameters associated with the field of view. This includes the “angle_bl,” “angle_al,” “angle_br,” and “angle_ar” parameters. These parameters define the angular range in front of the vehicle, which is where possible obstacles are considered. Defining these parameters establishes the limits of the field of view.

The “preprocess_lidar” function preprocesses the LiDAR data that is obtained from the LiDAR sensor. It filters out unnecessary data while ensuring that data from within the defined field of view are used for further processing. Since this processes LiDAR data within the specific field of view, this function directly contributes to accomplishing step 1.

The “find_max_gap” function recognizes the beginning and end indices of the largest gap in the LiDAR data. This function establishes a region in the LiDAR scan that is void of obstacles. Since this identifies the limits of the gap, it defines the field of view where the potential obstacles are not present, which does impact the process of determining where potential obstacles are.

The “lidar_callback” function processes LiDAR data to determine the obstacles in a specific angular range that is defined by the field of view parameters. The computation of “heading_angle” and processing of LiDAR data in this function contribute to discovering obstacles within the field of view.

The parameters in the “params.yaml” file that accomplish step 1 include the LiDAR simulation parameters which consists of the “scan_beams,” “scan_field_of_view,” and “scan_range” parameters. These parameters regulate the number of scan beams, angular field of view, and the maximum range of the LiDAR sensor. In addition, parameters that limit the speed and steering angle also accomplish step 1. This includes the “max_speed,” “max_steering_angle,” “max_accel,” “max_decel,” and “max_steering_vel” parameters. These parameters refine and limit the vehicles operational limits within the field of view. These parameters contribute to identifying the region in front of the vehicle where potential obstacles are detected.

Activity 3

The “preprocess_lidar” function plays a significant role in accomplishing the objective of step 2. It works by creating two arrays. One array represents the maximum distance for each angle, while the other array represents the sum of the minimum distance to an object. When an object descends within or at the predefined safe distance in the vehicle’s field of view (FOV), the function attributes a zero value to signify its presence as an immediate obstacle. This indicates that the object is within the safe distance of the vehicle and requires immediate attention to avoid a potential collision. On the other hand, returns beyond the safe distance are assigned non-zero values, indicating available space for the vehicle to navigate without encountering immediate obstacles. This process effectively distinguishes between areas that require immediate attention due to proximity and regions that are safe for navigation, thus enhancing the vehicle's navigation and obstacle avoidance capabilities. This feature is highly beneficial as it enables the vehicle to navigate through areas that are safe for movement while avoiding potential obstacles.

```
# Pre-process LiDAR data
def preprocess_lidar(self, ranges):

    data=[]
    data2=[]

    for i in range(self.ls_len_mod):
        if ranges[self.ls_str+i]<=self.safe_distance:
            data.append ([0,i*self.ls_ang_inc-self.angle_cen])
        elif ranges[self.ls_str+i]<=self.max_lidar_range:
            data.append ([ranges[self.ls_str+i],i*self.ls_ang_inc-self.angle_cen])
        else:
            data.append ([self.max_lidar_range,i*self.ls_ang_inc-self.angle_cen])

    k1 = 100
    k2 = 40

    for i in range(k1):

        s_range = 0

        for j in range(k2):
            index1 = int(i*len(ranges)/k1+j)
            if index1 >= len(ranges):
                index1 = index1-len(ranges)

            index2 = int(i*len(ranges)/k1-j)
            if index2 < 0:
                index2= index2 + len(ranges)

            s_range = s_range + min(ranges[index1],self.max_lidar_range)+ min(ranges[index2],self.max_lidar_range)

        data2.append(s_range)

    return np.array(data), np.array(data2)
```

Figure 3. “preprocess_lidar” Function in the “navigation.py” File

Activity 4

In activity 4, we completed the code for the “find_max_gap” function in the “navigation.py” file to achieve the objective of Step 3, which is to find the longest sequence of consecutive non-zero LiDAR returns among the free-space points. This can be seen in Figure 4 below.

Completed Lines of Code for the “find_max_gap” Function in the “navigation.py” File:

```
end_indx2=end_indx
```

```
str_indx2=str_indx
```

```
range_sum=range_sum_new
```

```
# Return the start and end indices of the maximum gap in free_space_ranges
def find_max_gap(self, proc_ranges):

    j=0
    str_indx=0;end_indx=0
    str_indx2=0;end_indx2=0

    range_sum = 0
    range_sum_new = 0

    for i in range (self.ls_len_mod):

        if proc_ranges[i,0]!=0:
            if j==0:
                str_indx=i
                range_sum_new = 0
                j=1
            range_sum_new = range_sum_new + proc_ranges[i,0]
            end_indx=i

        if j==1 and (proc_ranges[i,0]==0 or i==self.ls_len_mod-1):
            j=0

            if range_sum_new > range_sum:
                end_indx2= end_indx #added
                str_indx2= str_indx #added
                range_sum = range_sum_new #added

    return str_indx2, end_indx2
```

Figure 4. Completed Code for the “find_max_gap” Function in the “navigation.py” File

Activity 5

In activity 5, we completed the code for the “find_best_point” function in the “navigation.py” file to compute the desired direction of movement according to the formulation in equation 2 of the lab manual. This can be seen in Figure 5 below.

Completed Lines of Code for the “find_best_point” Function in the “navigation.py” File:

```
range_sum=0
best_heading=0
for i in range (start_1, end_i+1):
    range_sum=range_sum + proc_ranges[i, 0]
    best_heading=best_heading + proc_ranges[i, 0]*proc_ranges[i, 1]
    if range_sum!=0:
        best_heading=best_heading/range_sum
```

```
# start_i & end_i are the start and end indices of max-gap range, respectively
# Returns index of best (furthest point) in ranges
def find_best_point(self, start_i, end_i, proc_ranges):

    # added
    range_sum = 0
    best_heading = 0
    for i in range (start_i, end_i+1):

        range_sum = range_sum + proc_ranges[i,0]
        best_heading = best_heading + proc_ranges[i,0]*proc_ranges[i,1]

        if range_sum !=0:
            best_heading = best_heading/range_sum

    # added
    return best_heading
```

Figure 5. Completed Code for the “find_best_point” Function in the “navigation.py” File

The “find_best_point” function used to compute the desired direction of movement according to the formulation in equation 2 would be a better choice than the furthest point in the largest gap in equation 1 from the lab manual. This function offers various advantages compared to the method in equation 1. The “find_best_point” function incorporates data from all LiDAR beam angles within the largest gap to compute an average value. This facilitates a more comprehensive understanding of the surrounding environment of the vehicle. This function iterates through the LiDAR beam angles in the largest gap, and calculates the sum of the range measurements, which is expressed as “range_sum,” and the weighted sum of beam angles multiplied by their corresponding ranges, which is expressed as “best_heading.” By considering both the distance to obstacles and their distribution, we calculate the weighted average of the LiDAR beam angles to compute the desired direction of movement. In contrast, the formulation in equation 1 relies only on the furthest point within the largest gap, which uses data from only two LiDAR beam angles.

One angle corresponds to the start of the largest gap and the other angle corresponds to the end of the largest gap. As a result, relying on data from only two LiDAR beam angles can present inaccurate navigation decisions. Therefore, the “find_best_point” function according to the formulation in equation 2 would be a better choice than the furthest point in the largest gap in equation 1 from the lab manual.

Activity 6

Derivation of Optimization Formulation:

Let λ_i represent the Lagrange Multiplier for the Inequality Constraint, given by:

$$L(W, \lambda_i) = \frac{1}{2} W^T \cdot W + \lambda_i (W^T p_i + 1)$$

$$\frac{\delta L}{\delta W} = W + \lambda_i = 0$$

$$(-\lambda_i p_i)^T + 1 \leq 0$$

$$\lambda_i \geq -\frac{1}{||p_i||^2}$$

$$\frac{1}{2} (-\lambda_i p_i)^T \cdot (-\lambda_i p_i) = \frac{1}{2} \lambda_i^2 p_i^T p_i$$

$$\frac{1}{2} \lambda_i^2 p_i^T p_i ; \text{ such that } \lambda_i \geq -\frac{1}{||p_i||^2}$$

Therefore, the optimization problem is $\max_{\lambda_i} \frac{1}{2} \lambda_i^2 p_i^T p_i ; \text{ such that } \lambda_i \geq -\frac{1}{||p_i||^2}$.

Activity 7

The value of $0 \leq \alpha \leq 1$ affects the solution to the optimization problem in equation 8 from the lab manual. The value of the design parameter α , is responsible for controlling the impact of w_{k-1} , which is the degree of smoothing. The parameter α fine-tunes the balance between smooth barrier transitions and the responsiveness to any environmental changes.

When α is at the extreme value of $\alpha = 0$, equation 8 becomes $\min_{w_k} \left(\frac{1}{2} (w_k - w_{k-1})^2 \right)$:

$$\begin{aligned} \min_{w_k} \frac{1}{2} \alpha w_k^T w_k + \frac{1}{2} (1 - \alpha) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\ s. t. \\ \min_{w_k} \frac{1}{2} (0) w_k^T w_k + \frac{1}{2} (1 - 0) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\ s. t. \end{aligned}$$

$$\min_{w_k} \left(\frac{1}{2} (w_k - w_{k-1})^2 \right)$$

This means that the model disregards the historical barrier positions and prioritizes the clearance of obstacles. This leads to fast shifts in the barrier's location from one time step to the next.

When α is at the extreme value of $\alpha = 1$, equation 8 becomes $\min_{w_k} \frac{1}{2} w_k^T w_k$, which is the same as equation 7 from the lab manual:

$$\begin{aligned} \min_{w_k} \frac{1}{2} \alpha w_k^T w_k + \frac{1}{2} (1 - \alpha) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\ s. t. \\ \min_{w_k} \frac{1}{2} (1) w_k^T w_k + \frac{1}{2} (1 - 1) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\ s. t. \\ \min_{w_k} \frac{1}{2} (1) w_k^T w_k + \frac{1}{2} (0) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\ s. t. \\ \min_{w_k} \frac{1}{2} w_k^T w_k \\ s. t. \end{aligned}$$

This means that the model only focuses on the previous barrier's position to make sure that there are smooth transitions.

Activity 8

Showing that Equation 9 has the same Solution as the one in Equation 8:

$$\begin{aligned}
 \min_{w_k} & \frac{1}{2} \alpha w_k^T w_k + \frac{1}{2} (1 - \alpha) (w_k - w_{k-1})^T (w_k - w_{k-1}) \\
 & \quad s. t. \\
 & = \frac{1}{2} \alpha w_k^T w_k + \frac{1}{2} (1 - \alpha) w_k^T w_k - \frac{1}{2} (1 - \alpha) w_{k-1}^T w_k - \frac{1}{2} (1 - \alpha) w_k^T w_{k-1} \\
 & \quad + \frac{1}{2} (1 - \alpha) w_{k-1}^T w_{k-1} \\
 \min_{w_k} & \frac{1}{2} w_k^T w_k + (\alpha - 1) w_{k-1}^T w_k \\
 & \quad s. t.
 \end{aligned}$$

By expanding out the objective function of equation 8, we can see that it is a combination of the terms in equation 9 with the additional terms of w_{k-1} , which also takes the same form as equation 9. Therefore, equation 9 has the same solution as the one in equation 8, making them equivalent.

Activity 9

Derivation of IQP Formulation, $\min_{w, s.t.} \frac{1}{2} w^T w$:

Using $d = d_r + d_l = \frac{2}{\sqrt{w^T w}}$:

Gap, the distance between the two lines is given by:

$$d = d_r + d_l = \frac{2}{\sqrt{w^T w}}$$

$$d = \frac{2\sqrt{w^T w}}{w^T w}$$

$$\frac{\sqrt{w^T w}}{d} = \frac{w^T w}{2}$$

To maximize d , the right side of the equation must be minimized:

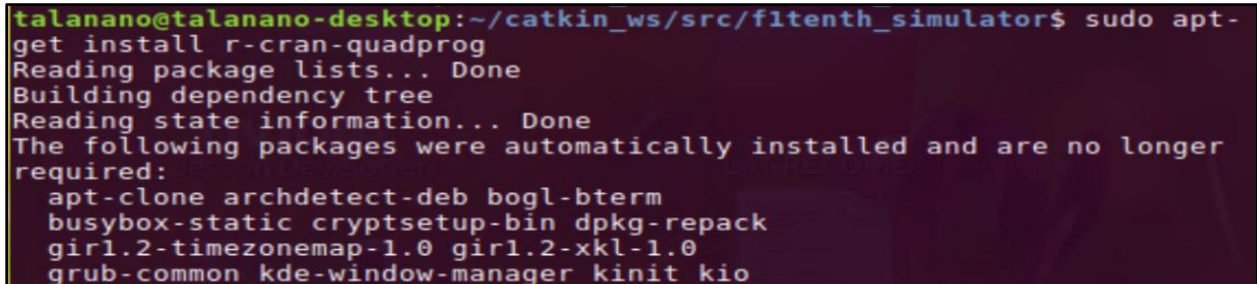
$$\begin{aligned}
 \min_w & \frac{w^T w}{2} \\
 & \quad s. t.
 \end{aligned}$$

$$\begin{aligned}
& \min_w \frac{1}{2} w^T w \\
& \text{s. t.} \\
& w^T p_i + b - 1 \geq 0 \\
& w^T p_i + b + 1 \leq 0 \\
& -1 + \epsilon \leq b \leq 1 - \epsilon
\end{aligned}$$

These additional constraints on the value of b are necessary. The rational for this is because these constraints define secure margins to the left and right of the vehicle, where ϵ acts as a buffer and is less than zero (a small number).

Activity 10

In activity 10, we installed the Python QP solver called “quadprog.” To do this, we used the “`sudo apt-get install r-cran-quadprog`” command. This can be seen in Figure 6 below.



```

talananano@talananano-desktop:~/catkin_ws/src/fltenth_simulator$ sudo apt-
get install r-cran-quadprog
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer
required:
  apt-clone archdetect-deb bogl-bterm
  busybox-static cryptsetup-bin dpkg-repack
  gir1.2-timexonemap-1.0 gir1.2-xkl-1.0
  grub-common kde-window-manager kinit kio

```

Figure 6. Running “`sudo apt-get install r-cran-quadprog`” Command at the Terminal

Activity 11

In activity 11, we considered the optimization formulations for obtaining the virtual line barriers in equations 9 and 20 from the lab manual, and the standard form for the “quadprog” in equation 21 from the lab manual. Then, we identified the corresponding variables. Following this, we completed the missing code for the “getWalls” function in the “navigation.py” file to find the solution for the problem in equation 9 using the “quadprog” solver. The code that would provide a solution to the problem in equation 9 is inside the “if self.optim_mode==0” statement. This can be seen in Figure 7 below.

Completed Lines of Code for the “getWalls” Function in the “navigation.py” File:

```
Pr=np.array([[1.0, 0],[0, 1.0]])
```

```
Pl=np.array([[1.0, 0],[0, 1.0]])
```

```
bl=np.full((self.n_pts_l),1.0,dtype=np.float64)
```

```
br=np.full((self.n_pts_r),1.0,dtype=np.float64)
```

```
Cl=-(left_obstacles.T)
```

```
Cr=-(right_obstacles.T)
```

```
def getWalls(self, left_obstacles, right_obstacles, wl0, wr0, alpha):

    if self.optim_mode == 0:

        Pr = np.array([[1.0,0],[0,1.0]]) #added
        Pl = np.array([[1.0,0],[0,1.0]]) #added

        bl = np.full((self.n_pts_l),1.0,dtype=np.float64) #added
        br = np.full((self.n_pts_r),1.0,dtype=np.float64) #added

        Cl= -(left_obstacles.T) #added
        Cr= -(right_obstacles.T) #added
        al = (1-alpha)*wl0
        ar = (1-alpha)*wr0

        wl = solve_qp(Pl.astype(np.float), al.astype(np.float), Cl.astype(np.float), bl.astype(np.float), 0)[0]
        wr = solve_qp(Pr.astype(np.float), ar.astype(np.float), Cr.astype(np.float), br.astype(np.float), 0)[0]

    else:

        P = np.array([[1.0,0,0],[0,1.0,0],[0,0,0.0001]])

        bl = np.full((self.n_pts_l),1.0,dtype=np.float64) #added
        br = np.full((self.n_pts_r),1.0,dtype=np.float64) #added
        b = np.concatenate((br,bl,np.array([-0.99, -0.99])))

        Cl= -(left_obstacles.T) #added
        Cr= -(right_obstacles.T) #added
        C1 = np.vstack((-Cr,br))
        C2 = np.vstack((Cl,- bl))
        C =np.hstack((C1,C2))
        C =np.hstack((C,np.array([[0,0],[0,0],[1.0,-1.0]])))

        a = np.zeros(3)

        ws = solve_qp(P.astype(np.float), a.astype(np.float), C.astype(np.float), b.astype(np.float), 0)[0]

        wr = np.array([ws[0]/(ws[2]-1),ws[1]/(ws[2]-1)]) #added
        wl = np.array([ws[0]/(ws[2]+1),ws[1]/(ws[2]+1)]) #added

    return wl, wr
```

Figure 7. Completed Code for the “getWalls” Function in the “navigation.py” File

Activity 12

In activity 12, we considered the optimization formulation for obtaining the parallel virtual line barrier in equation 21 and the standard form for the “quadprog” in equation 15 from the lab manual. Then, we identified the corresponding variables. Following this, we completed the missing code for the “getWalls” function in the “navigation.py” file to find the solution for the problem in equation 20 using the “quadprog” solver. The code that would provide a solution to the problem in equation 20 is under the “else” statement. This can be seen in Figure 8 below.

Completed Lines of Code for the “getWalls” Function in the “navigation.py” File:

```
bl=np.full((self.n_pts_l),1.0,dtype=np.float64)
```

```
br=np.full((self.n_pts_r),1.0,dtype=np.float64)
```

```
Cl=-(left_obstacles.T)
```

```
Cr=-(right_obstacles.T)
```

```
wr=np.array([ws[0]/(ws[2]-1),ws[1]/(ws[2]-1)])
```

```
wl=np.array([ws[0]/(ws[2]+1),ws[1]/(ws[2]+1)])
```

```

def getWalls(self, left_obstacles, right_obstacles, wl0, wr0, alpha):

    if self.optim_mode == 0:

        Pr = np.array([[1.0,0],[0,1.0]]) #added
        Pl = np.array([[1.0,0],[0,1.0]]) #added

        bl = np.full((self.n_pts_l),1.0,dtype=np.float64) #added
        br = np.full((self.n_pts_r),1.0,dtype=np.float64) #added

        Cl= -(left_obstacles.T) #added
        Cr= -(right_obstacles.T) #added
        al = (1-alpha)*wl0
        ar = (1-alpha)*wr0

        wl = solve_qp(Pl.astype(np.float), al.astype(np.float), Cl.astype(np.float), bl.astype(np.float), 0)[0]
        wr = solve_qp(Pr.astype(np.float), ar.astype(np.float), Cr.astype(np.float), br.astype(np.float), 0)[0]

    else:

        P = np.array([[1.0,0,0],[0,1.0,0],[0,0,0.0001]])

        bl = np.full((self.n_pts_l),1.0,dtype=np.float64) #added
        br = np.full((self.n_pts_r),1.0,dtype=np.float64) #added
        b = np.concatenate((br,bl,np.array([-0.99, -0.99])))

        Cl= -(left_obstacles.T) #added
        Cr= -(right_obstacles.T) #added
        C1 = np.vstack((-Cr,br))
        C2 = np.vstack((Cl,- bl))
        C =np.hstack((C1,C2))
        C =np.hstack((C,np.array([[0,0],[0,0],[1.0,-1.0]])))

        a = np.zeros(3)

        ws = solve_qp(P.astype(np.float), a.astype(np.float), C.astype(np.float), b.astype(np.float), 0)[0]

        wr = np.array([ws[0]/(ws[2]-1),ws[1]/(ws[2]-1)]) #added
        wl = np.array([ws[0]/(ws[2]+1),ws[1]/(ws[2]+1)]) #added

    return wl, wr

```

Figure 8. Completed Code for the “getWalls” Function in the “navigation.py” File

Activity 13

In activity 13, we completed the missing parts in equations 22 to 29 from the lab manual. Then, we completed the corresponding code in the “navigation.py” file. This can be seen in Figure 9 below.

Equation 22: Shown in Figure 9

$$d_l = 1/\text{math.sqrt}(\text{np.dot}(wl, T, wl))$$

$$d_r = 1/\text{math.sqrt}(\text{np.dot}(wr, T, wr))$$

Equation 23: Shown in Figure 9

$$w_r = w_r d_r$$

Equation 24: Shown in Figure 9

$$w_l = w_l d_l$$

Completed Lines of Code in “navigation.py” File:

```
dl=1/math.sqrt(np.dot(wl.Twl))
```

```
dr=1/math.sqrt(np.dot(wr.Twr))
```

```
wl_h=wl*dl
```

```
wr_h=wr*dr
```

```
dl = 1/math.sqrt(np.dot(wl.Twl)) #added
dr = 1/math.sqrt(np.dot(wr.Twr)) #added

wl_h = wl*dl #added
wr_h = wr*dr #added
```

Figure 9. Completed Code in the “navigation.py” File

Activity 14

Increasing the zero velocity, v_{s0} , increases the maximum value of v_s . Moreover, increasing the “d_stop” parameter makes the vehicle slow down faster as there is more “d_stop” that is encountered. The parameter “d_dec” makes the vehicle goes down slower, whereas increasing “d_min” makes the vehicle increase in speed as it goes farther away from the “d_stop” parameter. The plot of the vehicle velocity, v_s , as a function of d_{min} can be seen below in Figure 10.

Parameters to give $v_s = 0.3(1 - e^{-(d_{min}-0.6)/1})$:

Velocity zero = 0.3

“d_min” = The minimum distance = x

“d_stop” = stop_distance = 0.6

“d_dec” = stop_distance_decay = 1

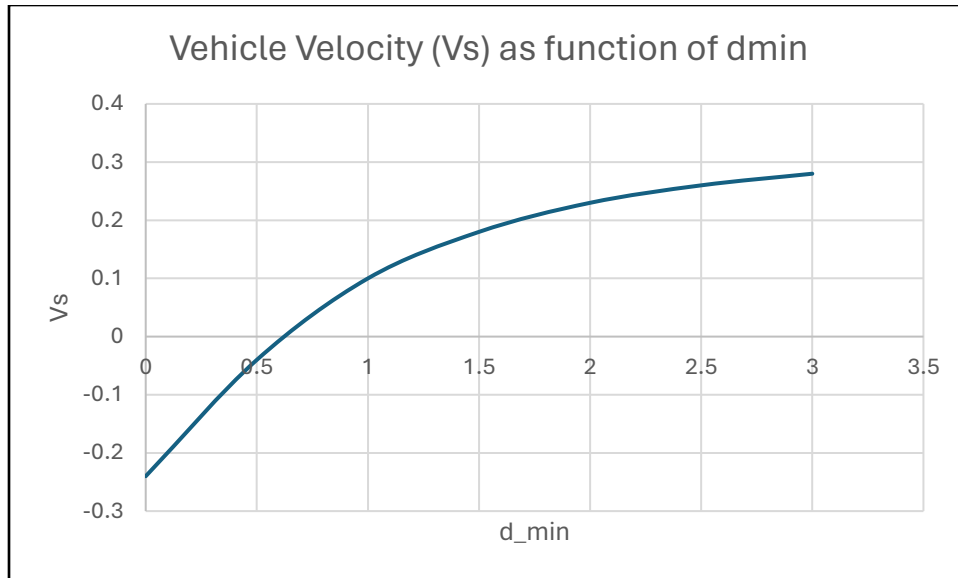


Figure 10. Plot of the Vehicle Velocity, v_s , as function of d_{min}

Activity 15

When the car initiates in normal mode, it proceeds forward until it detects an obstacle, such as a wall. At this point, it switches to turn mode, maneuvering at a turning velocity. If the car rotates sufficiently to bypass the obstacle, it reverts to normal mode until encountering another obstruction. However, if the obstacle cannot be avoided solely by turning, the car transitions from turn mode to backup mode, reversing its direction. After moving a distance away from the obstacle, it reverts to turn mode to attempt avoidance once more. If additional turning is required, the car returns to backup mode before transitioning back to turn mode. Once the necessary turning angle is achieved, the car switches back to normal mode. The finite-state machine diagram of the vehicle's operation is seen in Figure 11 below.

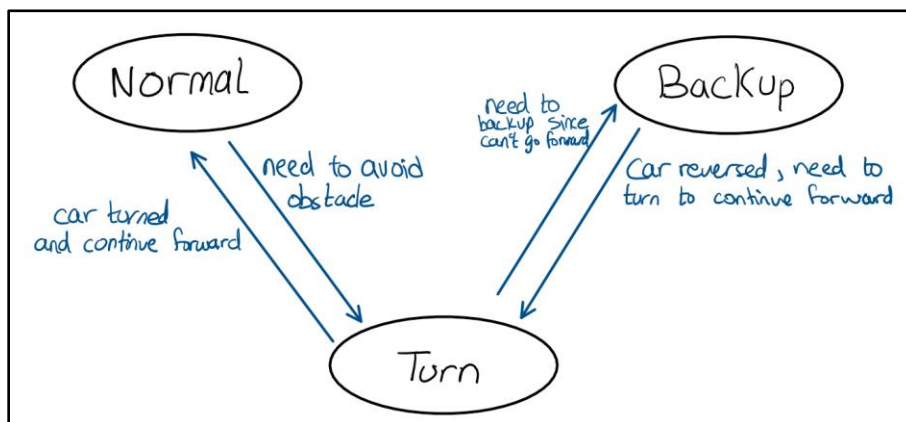


Figure 11. Finite-State Machine Diagram of Vehicle Operation

Activity 16

In Activity 16, we launched the 'simulator.launch' file and added a display markers. Then we changed the parameters so that the vehicle work as we expect in the rviz simulation environment. Where the markers represent separation barriers to the vehicle in the environment. While using both formulations, we were able to get the vehicle to pass the marker boxes. This can be seen in Figure 12 below.

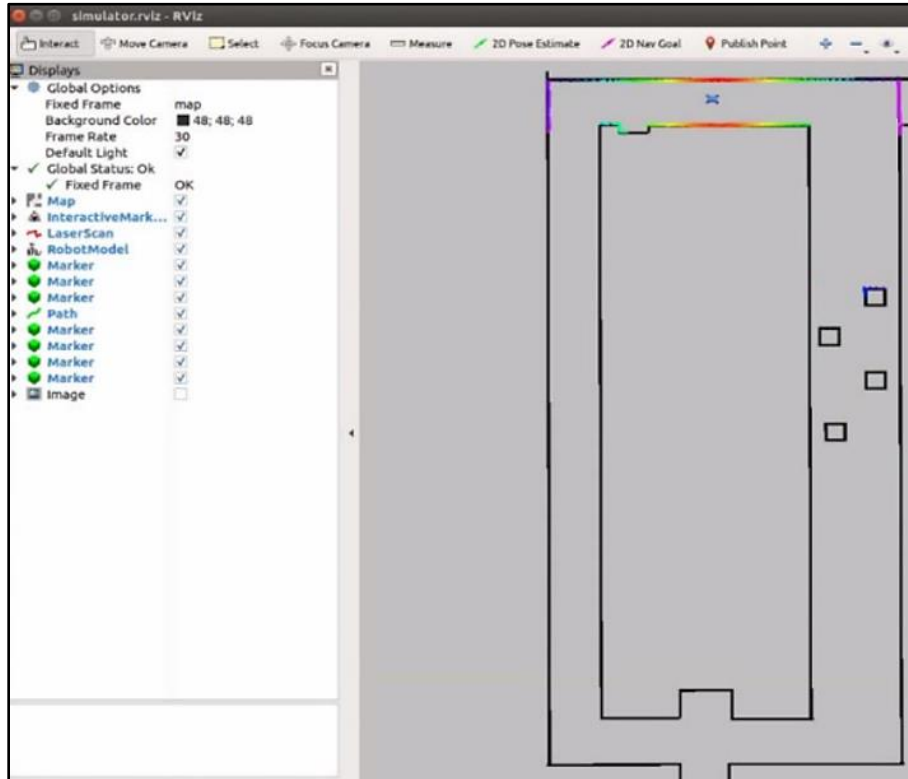


Figure 12. Markers in the RVIZ Simulation

Activity 17

A few key observations were made in the performance of the self-driving algorithm using the two optimization formulations in equations 9 and 20. We observed the impact of the “k_p,” “k_d,” “n_{pts_l},” “n_{pts_r},” and “safe_distance” parameters on the vehicle self-driving performance. The optimization formulations proved successful in navigating the obstacles presented in the simulation. Of the two formulations, equation 9 was found to be more effective for turning, while equation 20 was more suitable for driving in a straight line. When increasing the values of “n_{pts_l}” and “n_{pts_r}” the accuracy of the driving was improved, while decreasing these values resulted in some errors. We observed that decreasing the “safe_distance” increased the vehicle’s turning capacity and decreased its speed, whereas adjusting the values of “k_p” and “k_d” affected the speed and turning of the vehicle. Overall, these findings suggest that carefully selecting the appropriate values of the parameters in the optimization formulations can significantly improve the performance of the vehicle in navigating obstacles.

Activity 18

In activity 18, after we were satisfied with the self-driving controller’s performance in the simulation, we tested it on the actual vehicle. Before beginning, we set the value of “optim_mode” to select the correct optimization mode for our experiment. Then, we launched the “experiment.launch” file. To activate and deactivate self-driving mode, we toggled “RB” on the joystick. To activate the manual driving mode, we pressed “LB” on the joystick. We also ensured that the vehicle was always within our reach to make sure the connection between the vehicle and the joystick remained.

Activity 19

In activity 19, we adjusted the controller parameters to obtain the desired response. The parameters under consideration encompass the stopping distance and the velocity of the vehicle. Through the manipulation of these variables, we succeeded in enhancing the vehicle's capacity to self-correct, which allowed for a more consistent trajectory. From this experiment, we observed that when in independent mode, “optim_mode = 0”, the performance of the vehicle was enhanced in a narrower hallway, whereas in parallel mode, “optim_mode = 1”, the performance was more efficient in a wider hallway. In addition, the independent mode was observed to produce slightly more efficient turns compared to parallel mode.

Activity 20

The self-driving control algorithm functioned as anticipated, enabling the car to operate independently and avoid collisions. Additionally, the car was able to reverse and maintain its course. Nevertheless, the car occasionally reached a point where it would stop and commence turning its wheels left and right, preventing it from reversing. In such instances, manual intervention was required to drive the car and then enable the self-driving mode again to rectify the issue. Improving this feature would be significant to improve the car's operations and enhance its capabilities. Furthermore, increasing the car's speed would be beneficial since the present speed had to be reduced to facilitate the turning process.